```
class SoldOutConcertsController < ApplicationController
  def show
    concerts = Concert.includes(:venue, gigs: :band).all
    sold_out_concert_ids = concerts.select(&:sold_out?).map(&:id)
    render(json: {sold_out_concert_ids: sold_out_concert_ids})
  end
end
```

Here, we're returning a JSON file to be parsed on the client with the IDs of all the sold-out shows. (We could also send an updated list of tickets remaining, but we'll do that a different way using ActionCable in Chapter 9, *Immediate Communication with ActionCable*.) The JSON file has only one field, `sold_out_concert_ids`, which is an array of numbers.

When designing the client side, we have an immediate scaling issue to deal with. We could make each of our individual concerts a separate Stimulus controller that would each try to get this information. That might be fine if they were checking a global variable, but we probably don't want all of them making separate API calls to determine individual on-sale status. That's a lot of redundant calls to the sever.

This problem can be solved in a few different ways. What we're going to do here is create a Stimulus controller whose job it is to poll the server globally. That object will then change data attributes in the DOM, and we'll use Stimulus to pick up those data changes and update the display accordingly.

Our centralized object uses async/await to fetch our data—remember to run `rails stimulus:manifest:update` so the new controller is added to the esbuild bundle:

chapter_08/03/app/javascript/controllers/sold_out_data_controller.ts

```
import { Controller } from "@hotwired/stimulus"

export default class SoldOutDataController extends Controller {
  static targets = ["concert"]
  concertTargets: Array<HTMLElement>

  connect(): void {
    setInterval(() => this.updateData(), 1000 * 60)
```